

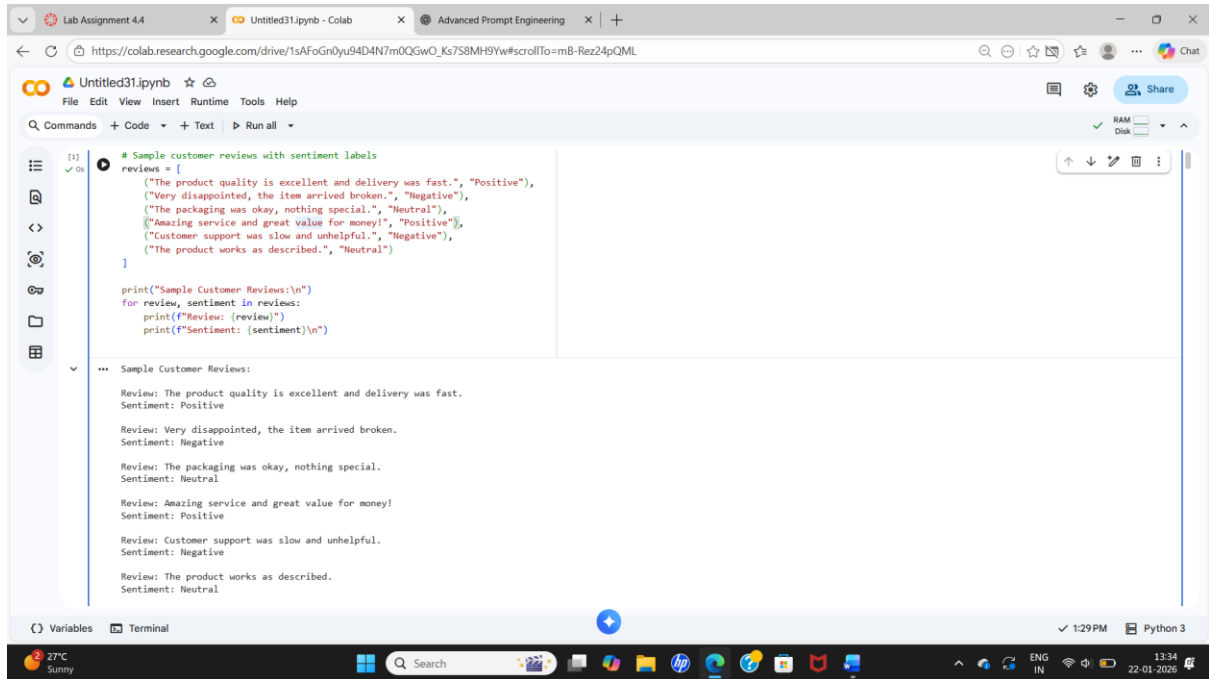
LAB ASSIGNMENT 4.4

NAME: R. PRANITHA REDDY

ID NO: 2303A52248

SUBJECT: AI ASST CODING

1.Scenario



The screenshot shows a Google Colab notebook titled 'Untitled31.ipynb'. The code defines a list of reviews with their sentiment labels. The output displays each review and its corresponding sentiment.

```
[1] ✓ On # Sample customer reviews with sentiment labels
reviews = [
    ("The product quality is excellent and delivery was fast.", "Positive"),
    ("Very disappointed, the item arrived broken.", "Negative"),
    ("The packaging was okay, nothing special.", "Neutral"),
    ("Amazing service and great value for money!", "Positive"),
    ("Customer support was slow and unhelpful.", "Negative"),
    ("The product works as described.", "Neutral")
]

print("Sample Customer Reviews:\n")
for review, sentiment in reviews:
    print(f"Review: {review}")
    print(f"Sentiment: {sentiment}\n")
```

Sample Customer Reviews:

Review: The product quality is excellent and delivery was fast.
Sentiment: Positive

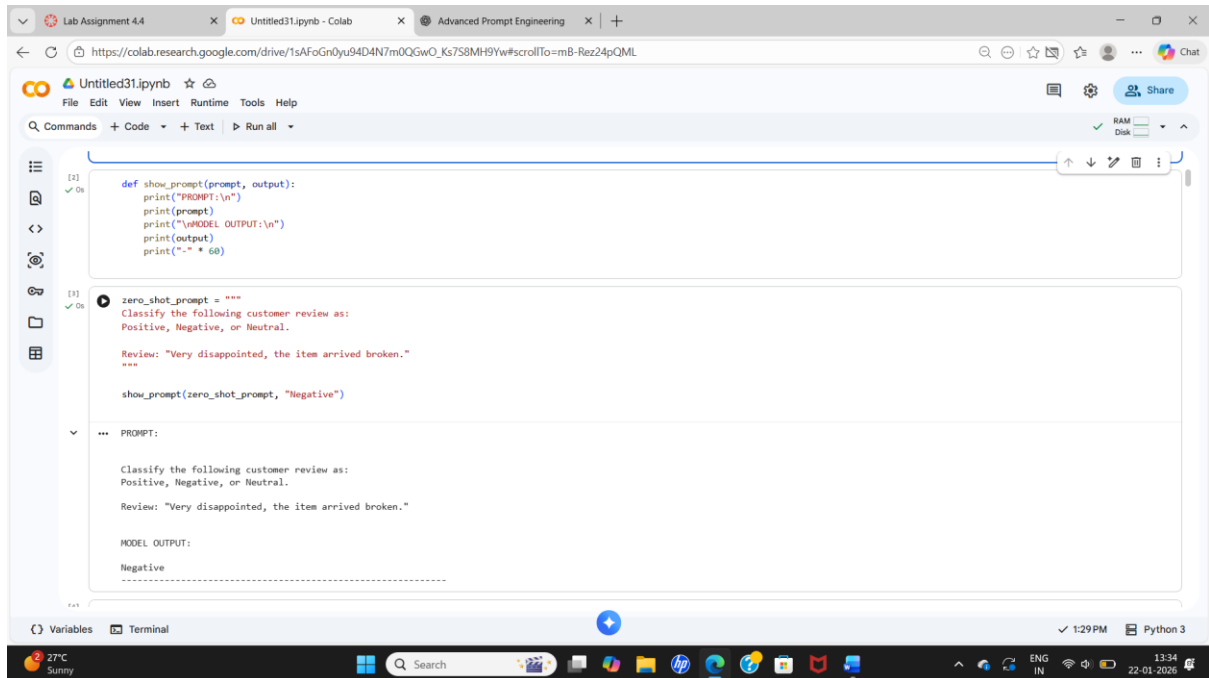
Review: Very disappointed, the item arrived broken.
Sentiment: Negative

Review: The packaging was okay, nothing special.
Sentiment: Neutral

Review: Amazing service and great value for money!
Sentiment: Positive

Review: Customer support was slow and unhelpful.
Sentiment: Negative

Review: The product works as described.
Sentiment: Neutral



The screenshot shows a Google Colab notebook titled 'Untitled31.ipynb'. The code defines a function to show prompts and a zero-shot prompt for sentiment classification. The output shows the prompt and the model's output.

```
[1] ✓ On def show_prompt(prompt, output):
print("PROMPT:\n")
print(prompt)
print("\nMODEL OUTPUT:\n")
print(output)
print("-" * 60)

[1] ✓ On zero_shot_prompt = """
Classify the following customer review as:
Positive, Negative, or Neutral.

Review: "Very disappointed, the item arrived broken."
"""

show_prompt(zero_shot_prompt, "Negative")
```

PROMPT:

Classify the following customer review as:
Positive, Negative, or Neutral.

Review: "Very disappointed, the item arrived broken."

MODEL OUTPUT:

Negative

The screenshot shows a Google Colab notebook interface. The top bar includes tabs for 'Lab Assignment 4.4', 'Untitled31.ipynb - Colab', and 'Advanced Prompt Engineering'. The notebook's title bar is 'Untitled31.ipynb' with a share button. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. The toolbar has 'Commands', '+ Code', '+ Text', and 'Run all'. The left sidebar shows icons for file explorer, search, and other tools. The main code area contains two code cells. The first cell, labeled '[4]', defines a prompt for sentiment classification using a single example (one-shot prompt) and a function `show_prompt`. The prompt text is: "Example: Review: 'The product quality is excellent and delivery was fast.' Sentiment: Positive. Now classify the following review: Review: 'Customer support was slow and unhelpful.'". The function call is `show_prompt(one_shot_prompt, "Negative")`. The second cell, labeled '[5]', defines a few-shot prompt with two examples. The prompt text is: "Example 1: Review: 'Amazing service and great value for money!' Sentiment: Positive. Example 2: Review: 'Very disappointed, the item arrived broken.' Sentiment: Negative. Now classify the following review: Review: 'The product works as described.'". The function call is `show_prompt(few_shot_prompt, "Neutral")`. The bottom status bar shows 'Variables', 'Terminal', '1:29 PM', and 'Python 3'. The system tray at the very bottom shows weather (27°C Sunny), search, and system icons.

```
[4]
one_shot_prompt = """
Example:
Review: "The product quality is excellent and delivery was fast."
Sentiment: Positive

Now classify the following review:
Review: "Customer support was slow and unhelpful."
"""

show_prompt(one_shot_prompt, "Negative")

PROMPT:

Example:
Review: "The product quality is excellent and delivery was fast."
Sentiment: Positive

Now classify the following review:
Review: "Customer support was slow and unhelpful."

MODEL OUTPUT:
Negative
-----

[5]
few_shot_prompt = """
Example 1:
Review: "Amazing service and great value for money!"
Sentiment: Positive

Example 2:
Review: "Very disappointed, the item arrived broken."
Sentiment: Negative

Now classify the following review:
Review: "The product works as described."
"""

show_prompt(few_shot_prompt, "Neutral")

PROMPT:

Example 1:
Review: "Amazing service and great value for money!"
Sentiment: Positive

Example 2:
Review: "Very disappointed, the item arrived broken."
Sentiment: Negative

Example 3:
Review: "The packaging was okay, nothing special."
Sentiment: Neutral

Now classify the following review:
Review: "The product works as described."

MODEL OUTPUT:
```

This screenshot shows the same Google Colab notebook interface as the first one, but with different code. The first code cell, labeled '[5]', defines a prompt for sentiment classification using three examples (few-shot prompt) and a function `show_prompt`. The prompt text is: "Example 2: Review: 'Very disappointed, the item arrived broken.' Sentiment: Negative. Example 3: Review: 'The packaging was okay, nothing special.' Sentiment: Neutral. Now classify the following review: Review: 'The product works as described.'". The function call is `show_prompt(few_shot_prompt, "Neutral")`. The second cell, labeled '[6]', defines a prompt for sentiment classification using three examples (few-shot prompt) and a function `show_prompt`. The prompt text is: "Example 1: Review: 'Amazing service and great value for money!' Sentiment: Positive. Example 2: Review: 'Very disappointed, the item arrived broken.' Sentiment: Negative. Example 3: Review: 'The packaging was okay, nothing special.' Sentiment: Neutral. Now classify the following review: Review: 'The product works as described.'". The function call is `show_prompt(few_shot_prompt, "Neutral")`. The bottom status bar shows 'Variables', 'Terminal', '1:29 PM', and 'Python 3'. The system tray at the very bottom shows weather (27°C Sunny), search, and system icons.

```
[5]
Example 2:
Review: "Very disappointed, the item arrived broken."
Sentiment: Negative

Example 3:
Review: "The packaging was okay, nothing special."
Sentiment: Neutral

Now classify the following review:
Review: "The product works as described."
"""

show_prompt(few_shot_prompt, "Neutral")

PROMPT:

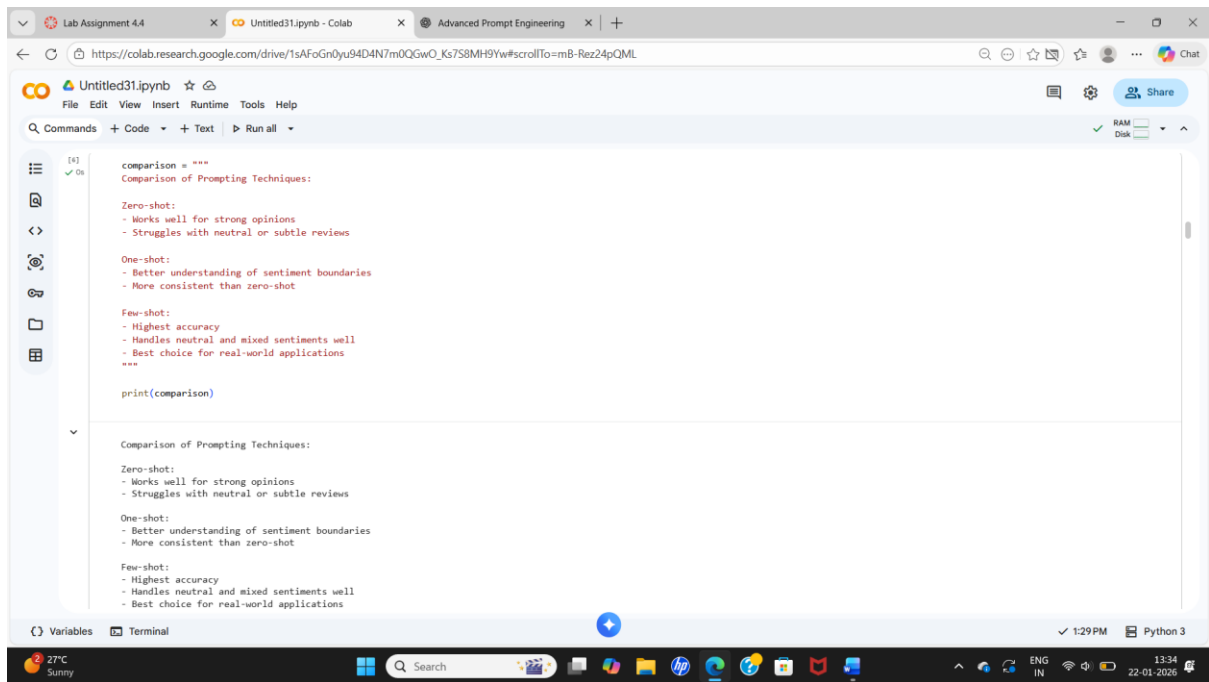
Example 1:
Review: "Amazing service and great value for money!"
Sentiment: Positive

Example 2:
Review: "Very disappointed, the item arrived broken."
Sentiment: Negative

Example 3:
Review: "The packaging was okay, nothing special."
Sentiment: Neutral

Now classify the following review:
Review: "The product works as described."

MODEL OUTPUT:
```



```
[4]: comparison = """
Comparison of Prompting Techniques:

Zero-shot:
- Works well for strong opinions
- Struggles with neutral or subtle reviews

One-shot:
- Better understanding of sentiment boundaries
- More consistent than zero-shot

Few-shot:
- Highest accuracy
- Handles neutral and mixed sentiments well
- Best choice for real-world applications
"""

print(comparison)
```

Comparison of Prompting Techniques:

Zero-shot:

- Works well for strong opinions
- Struggles with neutral or subtle reviews

One-shot:

- Better understanding of sentiment boundaries
- More consistent than zero-shot

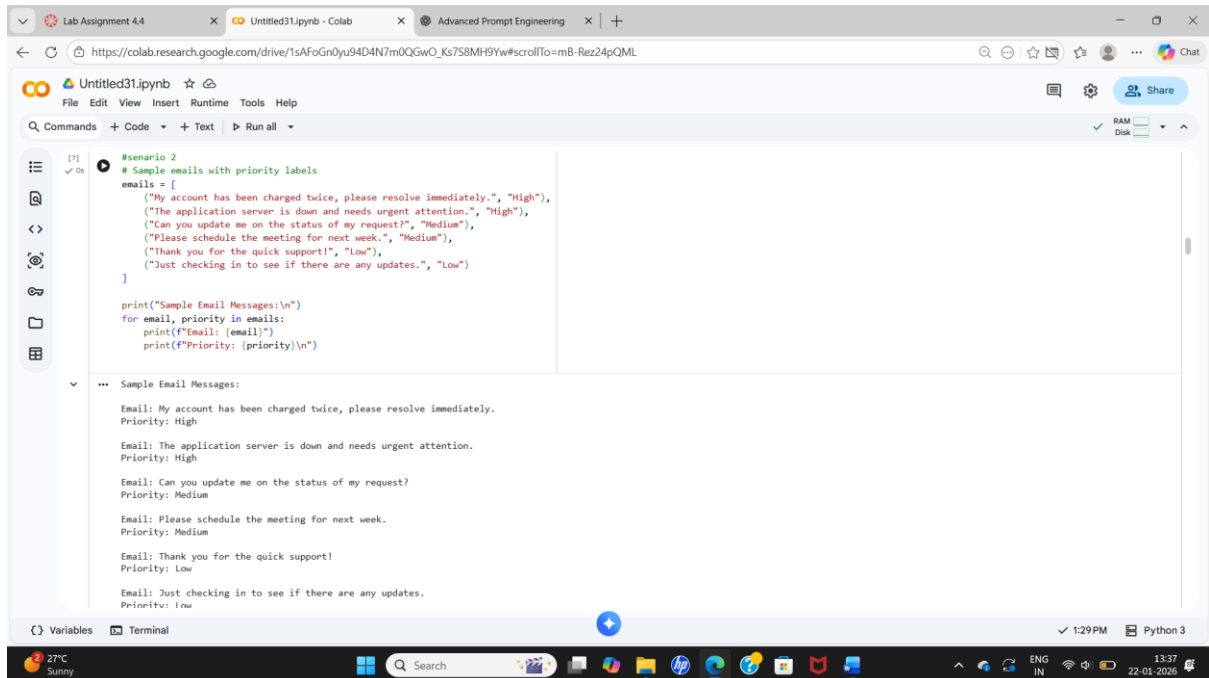
Few-shot:

- Highest accuracy
- Handles neutral and mixed sentiments well
- Best choice for real-world applications

FINAL CONCLUSION:

Few-shot prompting provides the best sentiment classification accuracy because multiple examples help the model understand sentiment patterns, tone, and neutrality more effectively than zero-shot or one-shot methods.

2.Scenario



```
[7] ✓ On #scenario 2
# Sample emails with priority labels
emails = [
    ("My account has been charged twice, please resolve immediately.", "High"),
    ("The application server is down and needs urgent attention.", "High"),
    ("Can you update me on the status of my request?", "Medium"),
    ("Please schedule the meeting for next week.", "Medium"),
    ("Thank you for the quick support!", "Low"),
    ("Just checking in to see if there are any updates.", "Low")
]

print("Sample Email Messages:\n")
for email, priority in emails:
    print(f"Email: {email}")
    print(f"Priority: {priority}\n")
```

Sample Email Messages:

Email: My account has been charged twice, please resolve immediately.
Priority: High

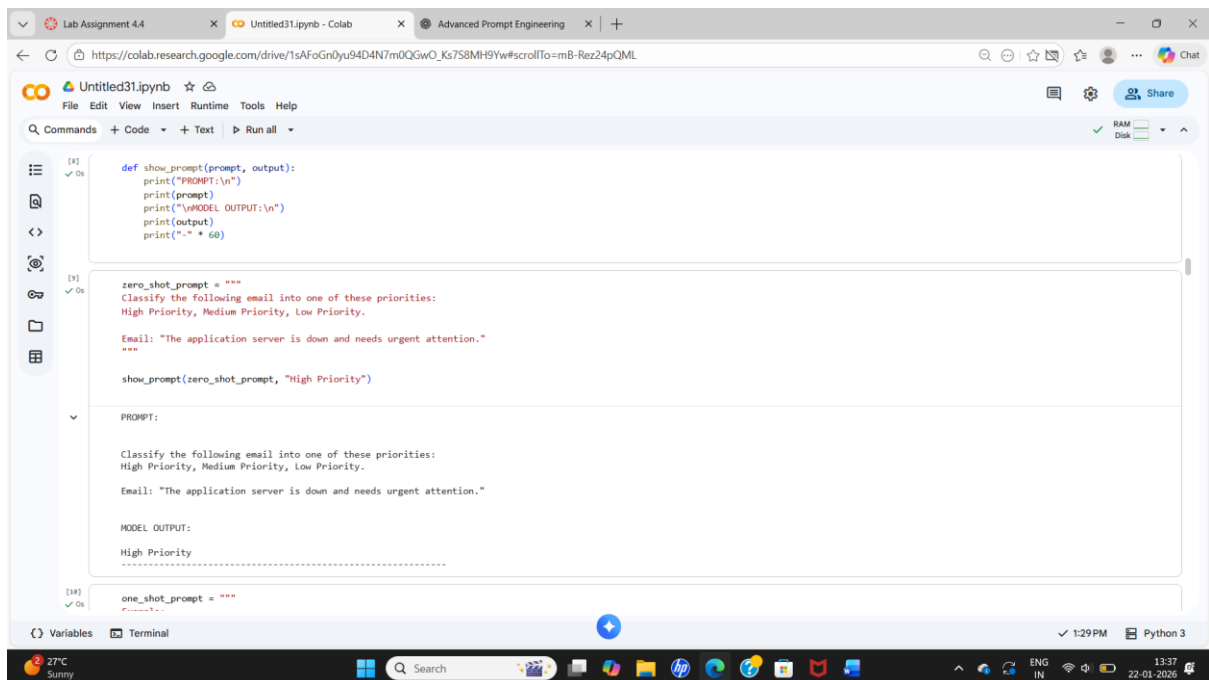
Email: The application server is down and needs urgent attention.
Priority: High

Email: Can you update me on the status of my request?
Priority: Medium

Email: Please schedule the meeting for next week.
Priority: Medium

Email: Thank you for the quick support!
Priority: Low

Email: Just checking in to see if there are any updates.
Priority: Low



```
[8] ✓ On def show_prompt(prompt, output):
    print("PROMPT:\n")
    print(prompt)
    print("\nMODEL OUTPUT:\n")
    print(output)
    print("-" * 60)
```

[9] ✓ On zero_shot_prompt = """
Classify the following email into one of these priorities:
High Priority, Medium Priority, Low Priority.

Email: "The application server is down and needs urgent attention."
"""

show_prompt(zero_shot_prompt, "High Priority")

PROMPT:

Classify the following email into one of these priorities:
High Priority, Medium Priority, Low Priority.

Email: "The application server is down and needs urgent attention."

MODEL OUTPUT:

High Priority

[10] ✓ On one_shot_prompt = """

Lab Assignment 4.4

Untitled31.ipynb - Colab

Advanced Prompt Engineering

+

https://colab.research.google.com/drive/1sAfoGn0yu94D4N7m0QGwO_Ks7S8MH9Yw#scrollTo=mB-Rez24pQML

Colab

Untitled31.ipynb

File Edit View Insert Runtime Tools Help

Commands Code + Text Run all

RAM Disk

[18] ✓ On

one_shot_prompt = """
Example:
Email: "My account has been charged twice, please resolve immediately."
Priority: High Priority

Now classify the following email:
Email: "Can you update me on the status of my request?"
"""

show_prompt(one_shot_prompt, "Medium Priority")

PROMPT:

Example:
Email: "My account has been charged twice, please resolve immediately."
Priority: High Priority

Now classify the following email:
Email: "Can you update me on the status of my request?"

MODEL OUTPUT:
Medium Priority

[11] ✓ On

few_shot_prompt = """
Example 1:
Email: "The application server is down and needs urgent attention."
Priority: High Priority

Example 2:
Email: "Please schedule the meeting for next week."
"""

Variables Terminal

1:29 PM Python 3

27°C Sunny

Search

hp

ENG IN

13:37

22-01-2026

Lab Assignment 4.4

Untitled31.ipynb - Colab

Advanced Prompt Engineering

+

https://colab.research.google.com/drive/1sAfoGn0yu94D4N7m0QGwO_Ks7S8MH9Yw#scrollTo=mB-Rez24pQML

Colab

Untitled31.ipynb

File Edit View Insert Runtime Tools Help

Commands Code + Text Run all

RAM Disk

[11] ✓ On

Example 2:
Email: "Please schedule the meeting for next week."
Priority: Medium Priority

Example 3:
Email: "Thank you for the quick support!"
Priority: Low Priority

Now classify the following email:
Email: "Just checking in to see if there are any updates."
"""

show_prompt(few_shot_prompt, "Low Priority")

PROMPT:

Example 1:
Email: "The application server is down and needs urgent attention."
Priority: High Priority

Example 2:
Email: "Please schedule the meeting for next week."
Priority: Medium Priority

Example 3:
Email: "Thank you for the quick support!"
Priority: Low Priority

Now classify the following email:
Email: "Just checking in to see if there are any updates."

MODEL OUTPUT:

Variables Terminal

1:29 PM Python 3

27°C Sunny

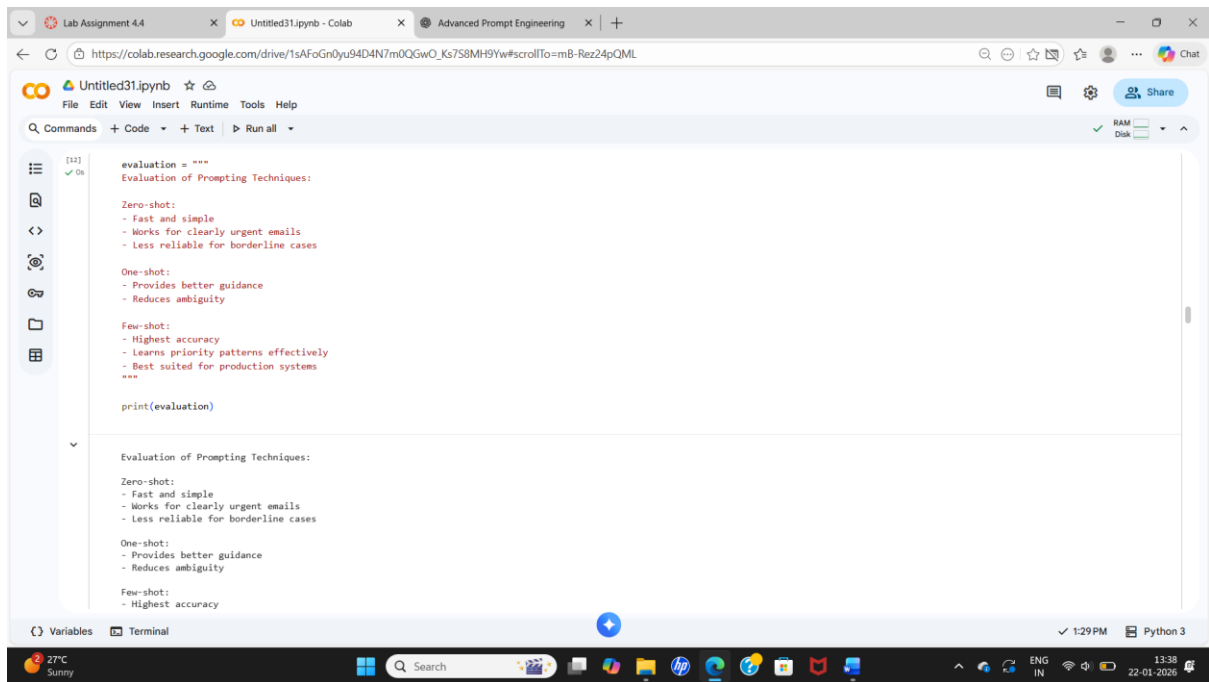
Search

hp

ENG IN

13:37

22-01-2026



The screenshot shows a Google Colab notebook interface. The browser tabs at the top include 'Lab Assignment 4.4', 'Untitled31.ipynb - Colab', and 'Advanced Prompt Engineering'. The notebook's address bar shows a Google Drive link. The notebook itself is titled 'Untitled31.ipynb' and has a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. The main code cell contains a Python script that defines a string variable 'evaluation' with a multi-line text block about prompting techniques, and then prints it. The output cell below shows the printed text. The bottom of the screen features a Windows taskbar with various icons, a search bar, and system information like '27°C Sunny', '1:29 PM', and '22-01-2026'.

```
evaluation = """
Evaluation of Prompting Techniques:

Zero-shot:
- Fast and simple
- Works for clearly urgent emails
- Less reliable for borderline cases

One-shot:
- Provides better guidance
- Reduces ambiguity

Few-shot:
- Highest accuracy
- Learns priority patterns effectively
- Best suited for production systems
"""

print(evaluation)
```

Evaluation of Prompting Techniques:

Zero-shot:

- Fast and simple
- Works for clearly urgent emails
- Less reliable for borderline cases

One-shot:

- Provides better guidance
- Reduces ambiguity

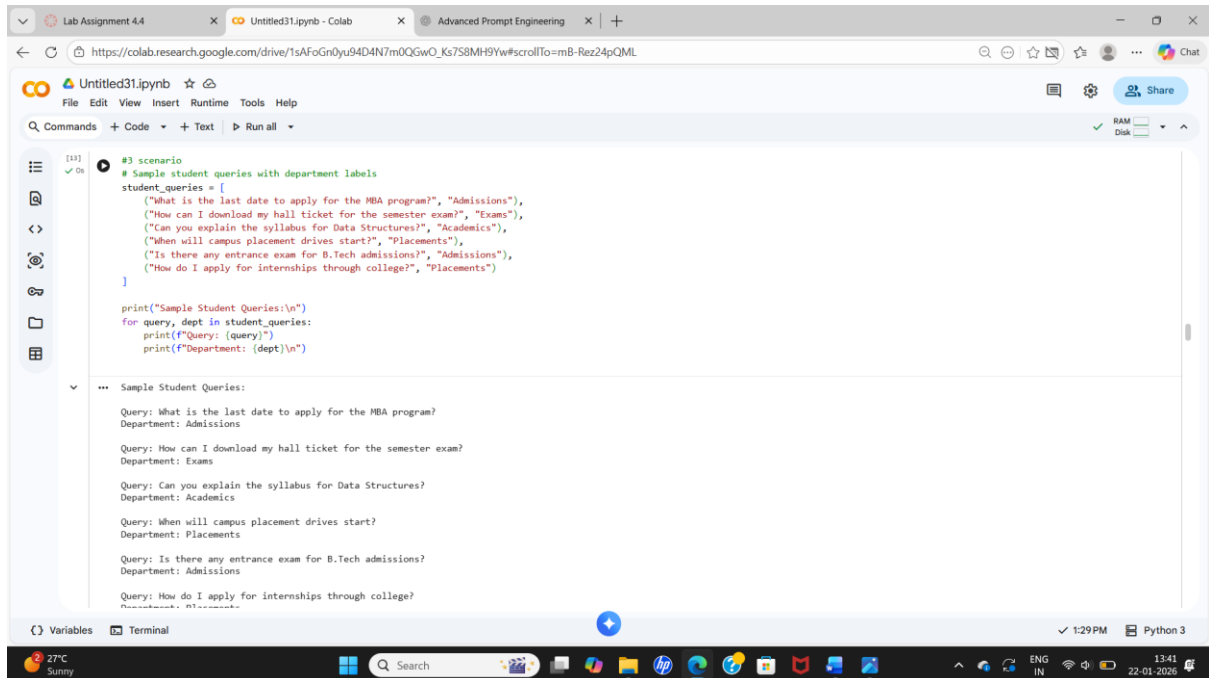
Few-shot:

- Highest accuracy

FINAL CONCLUSION:

Few-shot prompting produces the most reliable results because multiple examples help the model clearly distinguish between high, medium, and low urgency patterns in emails.

3.Scenario



The screenshot shows a Google Colab notebook titled 'Untitled31.ipynb'. The code defines a list of sample student queries and their corresponding departments. The output shows the queries and departments printed out.

```
[13]: #3 scenario
# Sample student queries with department labels
student_queries = [
    ("What is the last date to apply for the MBA program?", "Admissions"),
    ("How can I download my hall ticket for the semester exam?", "Exams"),
    ("Can you explain the syllabus for Data Structures?", "Academics"),
    ("When will campus placement drives start?", "Placements"),
    ("Is there any entrance exam for B.Tech admissions?", "Admissions"),
    ("How do I apply for internships through college?", "Placements")
]

print("Sample Student Queries:\n")
for query, dept in student_queries:
    print(f"Query: {query}")
    print(f"Department: {dept}\n")
```

Sample Student Queries:

Query: What is the last date to apply for the MBA program?
Department: Admissions

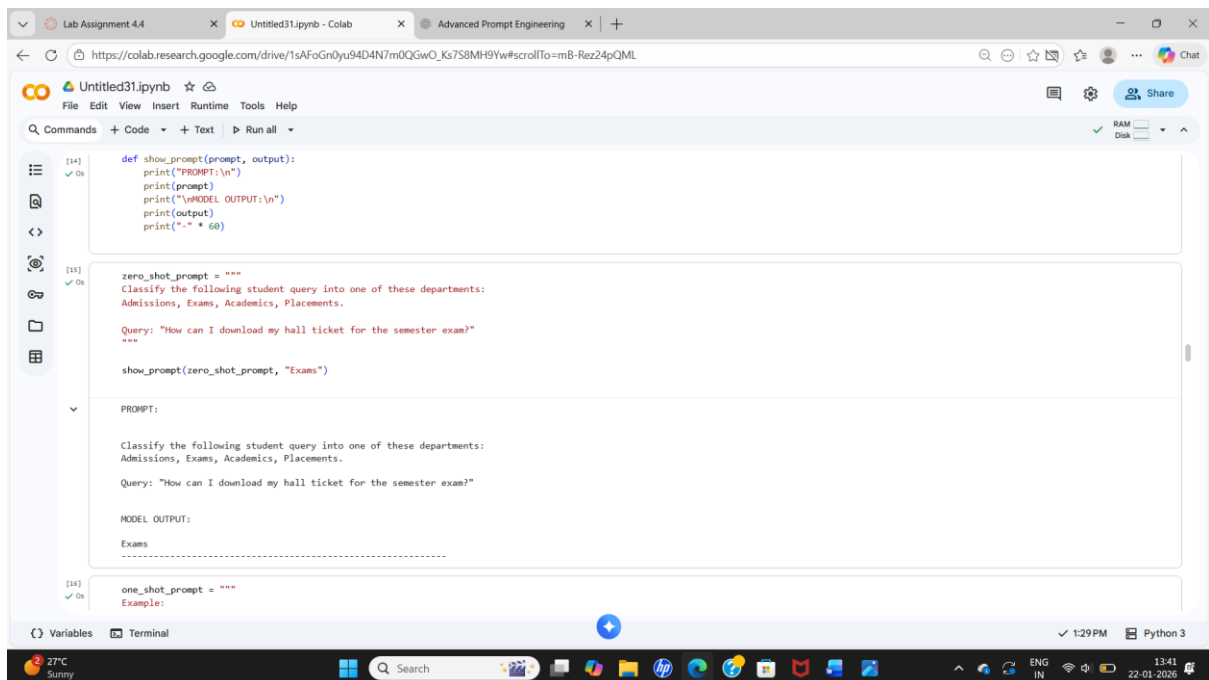
Query: How can I download my hall ticket for the semester exam?
Department: Exams

Query: Can you explain the syllabus for Data Structures?
Department: Academics

Query: When will campus placement drives start?
Department: Placements

Query: Is there any entrance exam for B.Tech admissions?
Department: Admissions

Query: How do I apply for internships through college?
Department: Placements



The screenshot shows a Google Colab notebook titled 'Untitled31.ipynb'. The code defines a function to show prompts and outputs, and a zero-shot prompt for classifying student queries into departments. The output shows the prompt and the model's output for a specific query.

```
[14]: def show_prompt(prompt, output):
print("PROMPT:\n")
print(prompt)
print("\nMODEL OUTPUT:\n")
print(output)
print("-" * 60)

[15]: zero_shot_prompt = """
Classify the following student query into one of these departments:
Admissions, Exams, Academics, Placements.

Query: "How can I download my hall ticket for the semester exam?"
"""

show_prompt(zero_shot_prompt, "Exams")

PROMPT:

Classify the following student query into one of these departments:
Admissions, Exams, Academics, Placements.

Query: "How can I download my hall ticket for the semester exam?"

MODEL OUTPUT:

Exams
-----

[16]: one_shot_prompt = """
Example:
```

Lab Assignment 4.4

Untitled31.ipynb - Colab

Advanced Prompt Engineering

+

https://colab.research.google.com/drive/1sAfoGn0yu94D4N7m0QGwO_Ks7S8MH9Yw#scrollTo=mB-Rez24pQML

Colab

Untitled31.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

[14] ✓ On

Query: "What is the last date to apply for the MBA program?"
Department: Admissions

Now classify the following query:
Query: "When will campus placement drives start?"

show_prompt(one_shot_prompt, "Placements")

PROMPT:

Example:
Query: "What is the last date to apply for the MBA program?"
Department: Admissions

Now classify the following query:
Query: "When will campus placement drives start?"

MODEL OUTPUT:
Placements

[17] ✓ On

few_shot_prompt = ""
Example 1:
Query: "What is the last date to apply for the MBA program?"
Department: Admissions

Example 2:
Query: "How can I download my hall ticket for the semester exam?"
Department: Exams

Variables Terminal

1:29 PM Python 3

27°C Sunny

Search

22-01-2026

Lab Assignment 4.4

Untitled31.ipynb - Colab

Advanced Prompt Engineering

+

https://colab.research.google.com/drive/1sAfoGn0yu94D4N7m0QGwO_Ks7S8MH9Yw#scrollTo=mB-Rez24pQML

Colab

Untitled31.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

[17] ✓ On

Example 3:
Query: "Can you explain the syllabus for Data Structures?"
Department: Academics

Example 4:
Query: "How do I apply for internships through college?"
Department: Placements

Now classify the following query:
Query: "Is there any entrance exam for B.Tech admissions?"

show_prompt(few_shot_prompt, "Admissions")

PROMPT:

Example 1:
Query: "What is the last date to apply for the MBA program?"
Department: Admissions

Example 2:
Query: "How can I download my hall ticket for the semester exam?"
Department: Exams

Example 3:
Query: "Can you explain the syllabus for Data Structures?"
Department: Academics

Example 4:
Query: "How do I apply for internships through college?"
Department: Placements

Now classify the following query:
Query: "Is there any entrance exam for B.Tech admissions?"

Variables Terminal

1:29 PM Python 3

27°C Sunny

Search

22-01-2026


```
Department: Exams

Example 3:
Query: "Can you explain the syllabus for Data Structures?"
Department: Academics

Example 4:
Query: "How do I apply for internships through college?"
Department: Placements

Now classify the following query:
Query: "Is there any entrance exam for B.Tech admissions?"

MODEL OUTPUT:
Admissions
-----

[14] ✓ On
analysis = """
Analysis of Prompting Techniques:

Zero-shot:
- Simple and fast
- May misclassify ambiguous queries

One-shot:
- Better guidance with one example
- Reduces confusion between departments

Few-shot:
- Highest accuracy
- Learns patterns from multiple examples
- Best suited for real chatbot deployment
"""
```

```
print(analysis)

Analysis of Prompting Techniques:

Zero-shot:
- Simple and fast
- May misclassify ambiguous queries

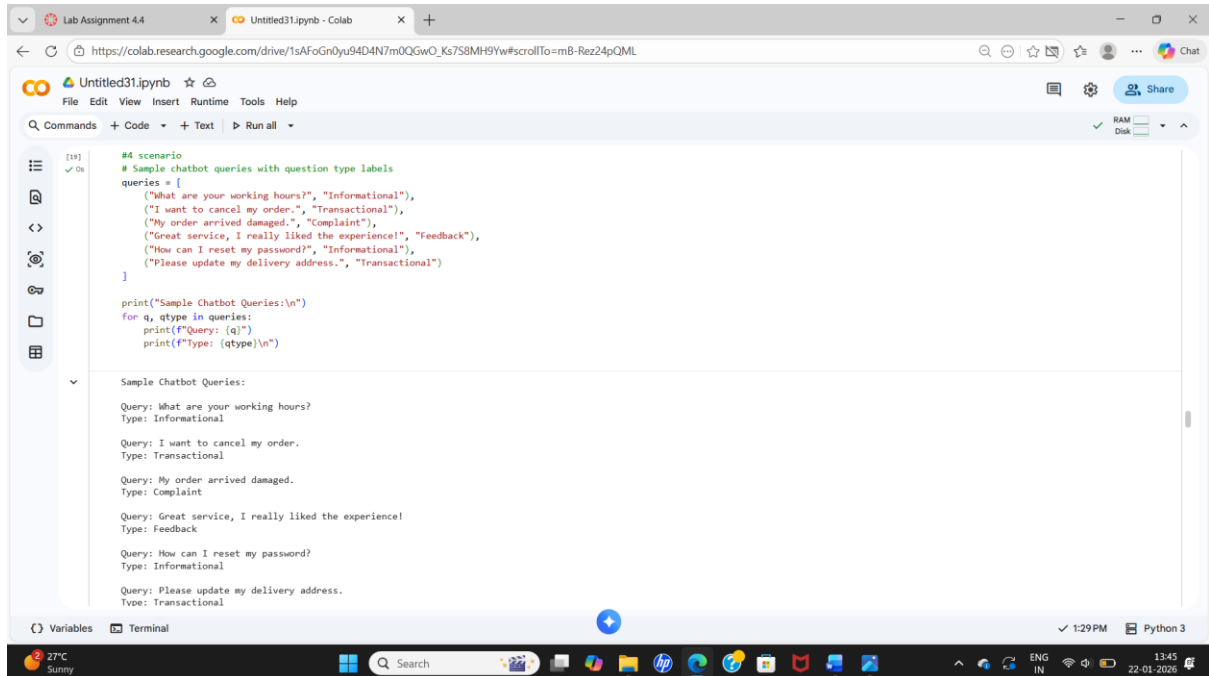
One-shot:
- Better guidance with one example
- Reduces confusion between departments

Few-shot:
- Highest accuracy
- Learns patterns from multiple examples
- Best suited for real chatbot deployment
```

FINAL CONCLUSION:

Few-shot prompting significantly improves student query routing accuracy by providing contextual patterns, making it the most effective approach for university chatbots.

4.Scenario



```
[19]: #4 scenario
# Sample chatbot queries with question type labels
queries = [
    ("What are your working hours?", "Informational"),
    ("I want to cancel my order.", "Transactional"),
    ("My order arrived damaged.", "Complaint"),
    ("Great service, I really liked the experience!", "Feedback"),
    ("How can I reset my password?", "Informational"),
    ("Please update my delivery address.", "Transactional")
]

print("Sample Chatbot Queries:\n")
for q, qtype in queries:
    print(f"Query: {q}")
    print(f"Type: {qtype}\n")
```

Sample Chatbot Queries:

Query: What are your working hours?
Type: Informational

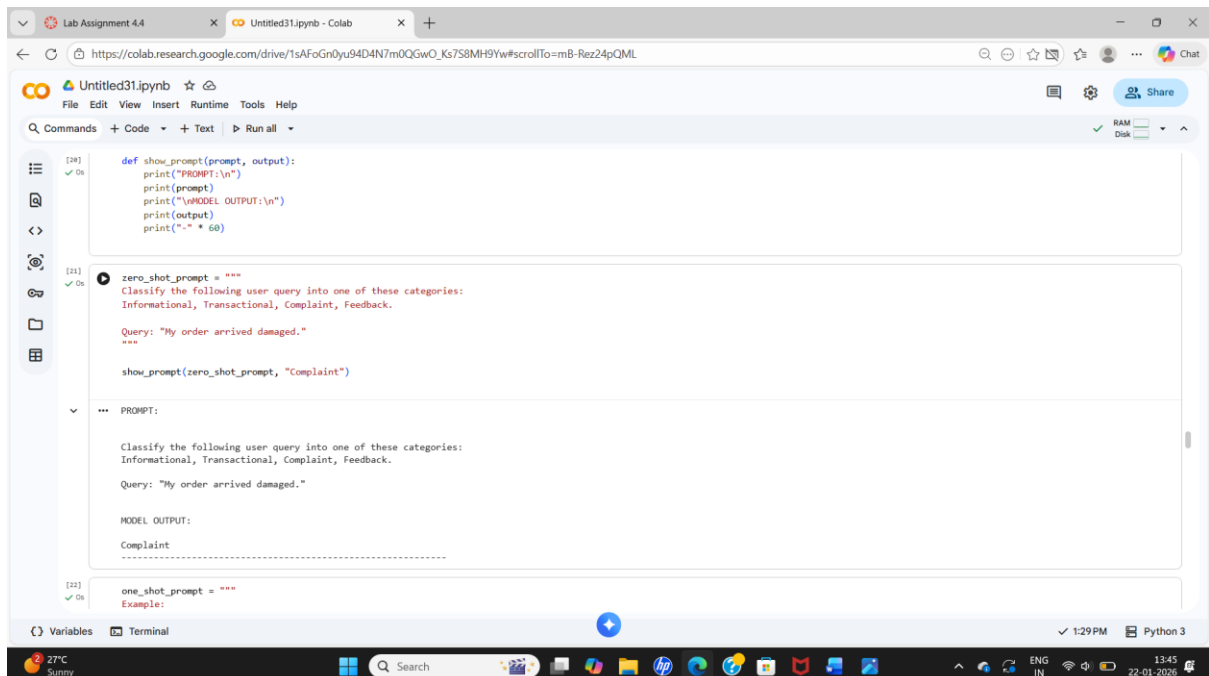
Query: I want to cancel my order.
Type: Transactional

Query: My order arrived damaged.
Type: Complaint

Query: Great service, I really liked the experience!
Type: Feedback

Query: How can I reset my password?
Type: Informational

Query: Please update my delivery address.
Type: Transactional



```
[20]: def show_prompt(prompt, output):
print("PROMPT:\n")
print(prompt)
print("\nMODEL OUTPUT:\n")
print(output)
print("-" * 60)

[21]: zero_shot_prompt = """
Classify the following user query into one of these categories:
Informational, Transactional, Complaint, Feedback.

Query: "My order arrived damaged."
"""

show_prompt(zero_shot_prompt, "Complaint")

--- PROMPT:

Classify the following user query into one of these categories:
Informational, Transactional, Complaint, Feedback.

Query: "My order arrived damaged."

MODEL OUTPUT:

Complaint

[22]: one_shot_prompt = """
Example:
```

The screenshot shows a Google Colab notebook interface. The browser tabs at the top include 'Lab Assignment 4.4' and 'Untitled31.ipynb - Colab'. The address bar shows a Google Drive link. The notebook has a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. On the left is a sidebar with icons for file management and execution. The main area contains two code cells. The first cell, labeled '[22] ✓ On', contains a query classification example using a one-shot prompt. The second cell, labeled '[23] ✓ On', contains a few-shot prompt example. The notebook status bar at the bottom shows 'Variables', 'Terminal', '1:29 PM', and 'Python 3'. The Windows taskbar at the very bottom shows the date '22-01-2026' and time '13:45'.

```
[22] ✓ On
Query: "Great service, I really liked the experience!"
Category: Feedback

Now classify the following query:
Query: "I want to cancel my order."
===

show_prompt(one_shot_prompt, "Transactional")

PROMPT:

Example:
Query: "Great service, I really liked the experience!"
Category: Feedback

Now classify the following query:
Query: "I want to cancel my order."

MODEL OUTPUT:
Transactional

[23] ✓ On
few_shot_prompt = """
Example 1:
Query: "What are your working hours?"
Category: Informational

Example 2:
Query: "Please update my delivery address."
Category: Transactional
```

This screenshot shows the same Google Colab notebook, but with a different code cell selected. The code cell, labeled '[23] ✓ On', demonstrates a few-shot prompt. It includes four examples of queries and their corresponding categories, followed by a new query to be classified. The notebook interface and status bar are identical to the previous screenshot.

```
[23] ✓ On
example >:
Query: "My order arrived damaged."
Category: Complaint

Example 4:
Query: "Great service, I really liked the experience!"
Category: Feedback

Now classify the following query:
Query: "How can I reset my password?"
===

show_prompt(few_shot_prompt, "Informational")

PROMPT:

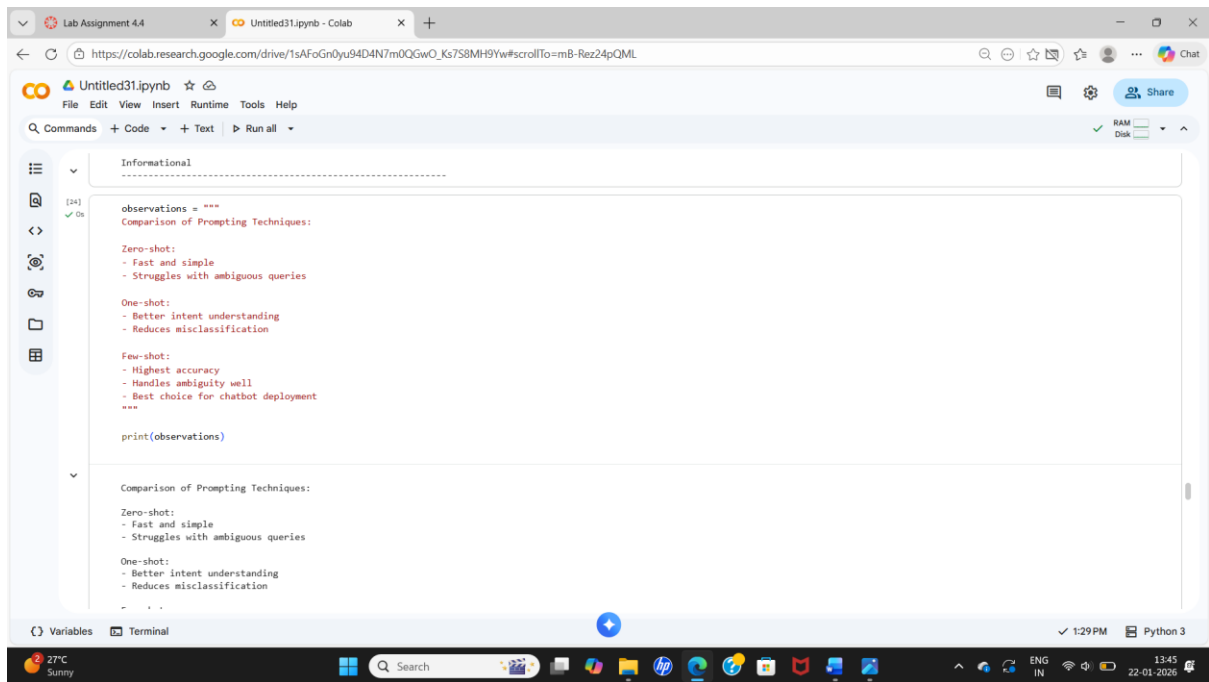
Example 1:
Query: "What are your working hours?"
Category: Informational

Example 2:
Query: "Please update my delivery address."
Category: Transactional

Example 3:
Query: "My order arrived damaged."
Category: Complaint

Example 4:
Query: "Great service, I really liked the experience!"
Category: Feedback

Now classify the following query:
Query: "How can I reset my password?"
```



```
observations = """
Comparison of Prompting Techniques:

Zero-shot:
- Fast and simple
- Struggles with ambiguous queries

One-shot:
- Better intent understanding
- Reduces misclassification

Few-shot:
- Highest accuracy
- Handles ambiguity well
- Best choice for chatbot deployment
"""

print(observations)
```

Comparison of Prompting Techniques:

Zero-shot:

- Fast and simple
- Struggles with ambiguous queries

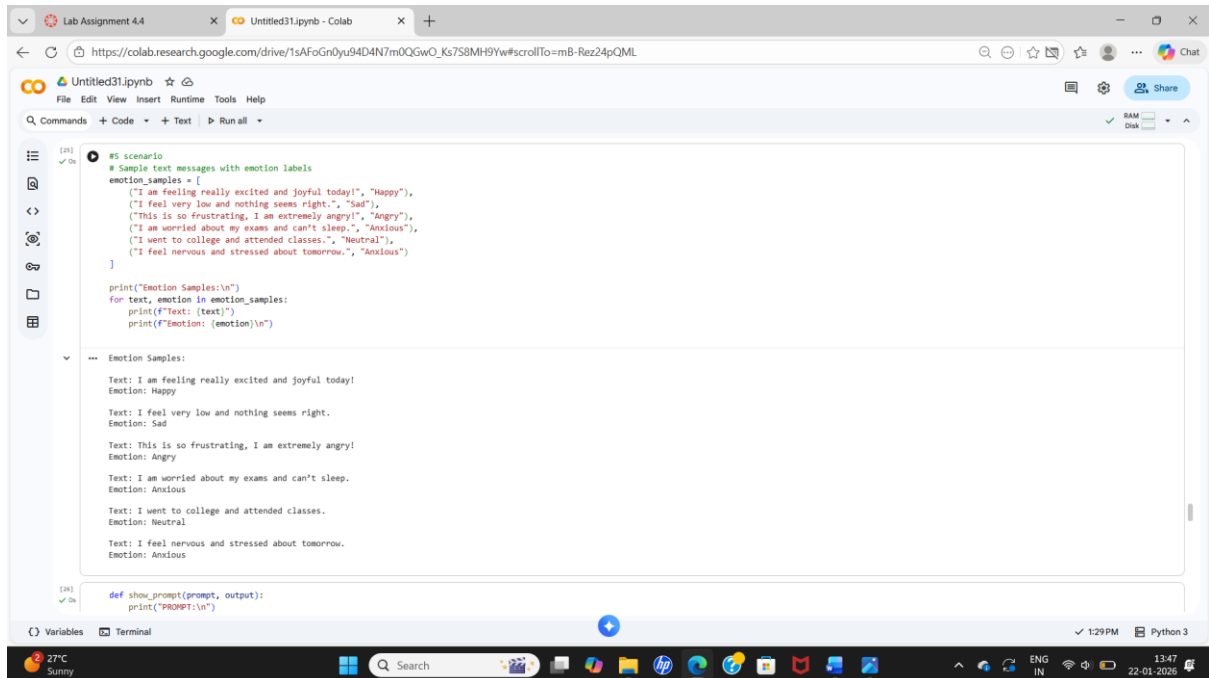
One-shot:

- Better intent understanding
- Reduces misclassification

FINAL CONCLUSION:

Few-shot prompting significantly improves chatbot question type detection by providing contextual examples, making it more robust and reliable than zero-shot and one-shot methods.

5.Scenario



```
[26] ✓ On #! scenario
# Sample text messages with emotion labels
emotion_samples = [
    ("I am feeling really excited and joyful today!", "Happy"),
    ("I feel very low and nothing seems right.", "Sad"),
    ("This is so frustrating, I am extremely angry!", "Angry"),
    ("I am worried about my exams and can't sleep.", "Anxious"),
    ("I went to college and attended classes.", "Neutral"),
    ("I feel nervous and stressed about tomorrow.", "Anxious")
]

print("Emotion Samples:\n")
for text, emotion in emotion_samples:
    print(f"Text: {text}")
    print(f"Emotion: {emotion}\n")

[27] ✓ On def show_prompt(prompt, output):
    print("PROMPT:\n")
    print(prompt)
    print("\nMODEL OUTPUT:\n")
    print(output)
    print("-" * 60)
```

Emotion Samples:

Text: I am feeling really excited and joyful today!
Emotion: Happy

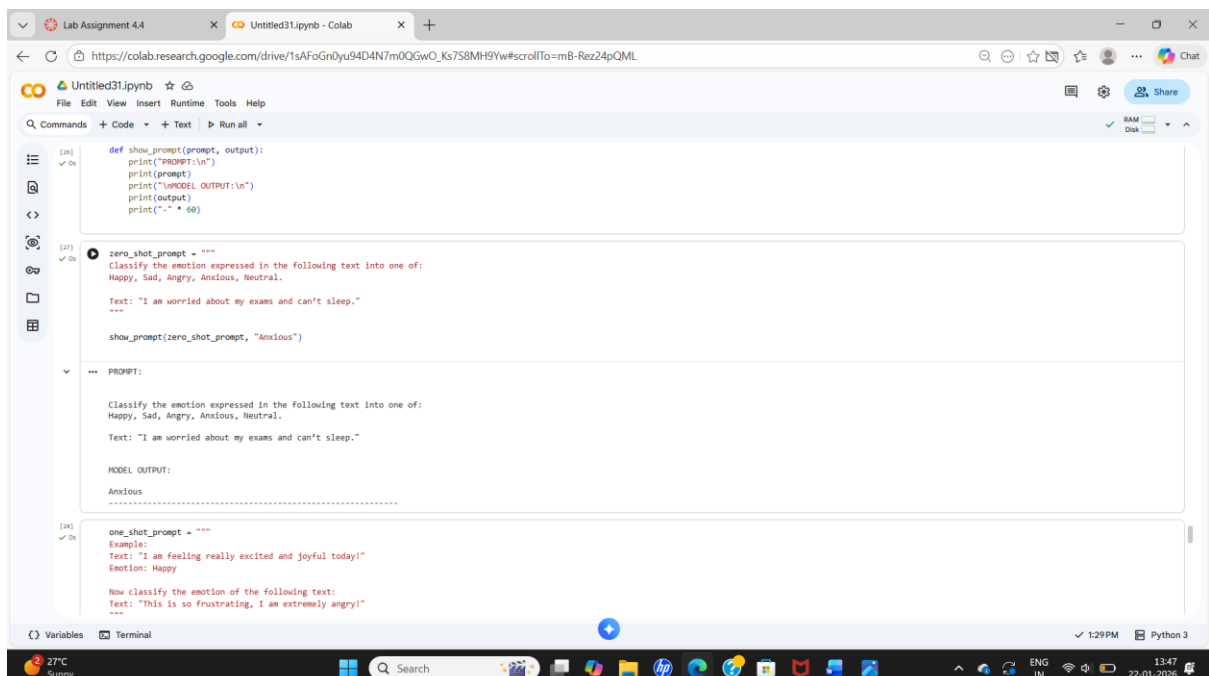
Text: I feel very low and nothing seems right.
Emotion: Sad

Text: This is so frustrating, I am extremely angry!
Emotion: Angry

Text: I am worried about my exams and can't sleep.
Emotion: Anxious

Text: I went to college and attended classes.
Emotion: Neutral

Text: I feel nervous and stressed about tomorrow.
Emotion: Anxious



```
[28] ✓ On def show_prompt(prompt, output):
    print("PROMPT:\n")
    print(prompt)
    print("\nMODEL OUTPUT:\n")
    print(output)
    print("-" * 60)

[29] ✓ On zero_shot_prompt = """
Classify the emotion expressed in the following text into one of:
Happy, Sad, Angry, Anxious, Neutral.

Text: "I am worried about my exams and can't sleep."
"""

show_prompt(zero_shot_prompt, "Anxious")

[30] ✓ On one_shot_prompt = """
Example:
Text: "I am feeling really excited and joyful today!"
Emotion: Happy

Now classify the emotion of the following text:
Text: "This is so frustrating, I am extremely angry!"
"""
```

PROMPT:

Classify the emotion expressed in the following text into one of:
Happy, Sad, Angry, Anxious, Neutral.

Text: "I am worried about my exams and can't sleep."

MODEL OUTPUT:

Anxious

Lab Assignment 4.4

Untitled31.ipynb - Colab

https://colab.research.google.com/drive/1sAfoGn0yu94D4N7m0QGwO_Ks7S8MH9Yw#scrollTo=mB-Rez24pQML

Colab

File Edit View Insert Runtime Tools Help

Commands + Code + Text + Run all

RAM Disk

[28] ✓ In

Now classify the emotion of the following text:
Text: "This is so frustrating, I am extremely angry!"

show_prompt(one_shot_prompt, "Angry")

PROMPT:

Example:
Text: "I am feeling really excited and joyful today!"
Emotion: Happy

Now classify the emotion of the following text:
Text: "This is so frustrating, I am extremely angry!"

MODEL OUTPUT:
Angry
.....

+ Code + Text

[29] ✓ In

few_shot_prompt = ""
Example 1:
Text: "I am feeling really excited and joyful today!"
Emotion: Happy

Example 2:
Text: "I feel very low and nothing seems right."
Emotion: Sad

Example 3:
Text: "This is so frustrating, I am extremely angry!"
Emotion: Angry

Example 4:
Text: "I am worried about my exams and can't sleep."
Emotion: Anxious

Variables Terminal

1:29 PM Python 3

27°C Sunny

Search

22-01-2026

Lab Assignment 4.4

Untitled31.ipynb - Colab

https://colab.research.google.com/drive/1sAfoGn0yu94D4N7m0QGwO_Ks7S8MH9Yw#scrollTo=mB-Rez24pQML

Colab

File Edit View Insert Runtime Tools Help

Commands + Code + Text + Run all

RAM Disk

[29] ✓ In

Example 5:
Text: "I went to college and attended classes."
Emotion: Neutral

Now classify the emotion of the following text:
Text: "I feel nervous and stressed about tomorrow."

show_prompt(few_shot_prompt, "Anxious")

PROMPT:

Example 1:
Text: "I am feeling really excited and joyful today!"
Emotion: Happy

Example 2:
Text: "I feel very low and nothing seems right."
Emotion: Sad

Example 3:
Text: "This is so frustrating, I am extremely angry!"
Emotion: Angry

Example 4:
Text: "I am worried about my exams and can't sleep."
Emotion: Anxious

Example 5:
Text: "I went to college and attended classes."
Emotion: Neutral

Now classify the emotion of the following text:
Text: "I feel nervous and stressed about tomorrow."

MODEL OUTPUT:
Anxious
.....

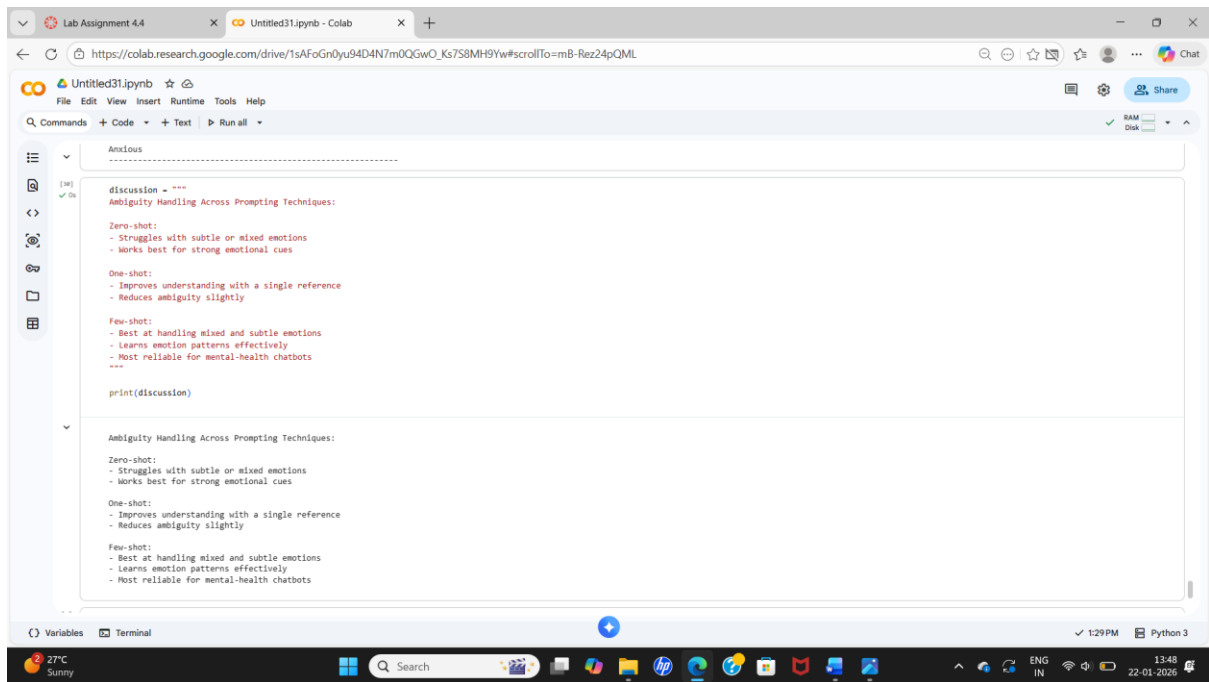
Variables Terminal

1:29 PM Python 3

27°C Sunny

Search

22-01-2026



FINAL CONCLUSION:

Few-shot prompting provides the highest accuracy in emotion detection because multiple labelled examples help the model distinguish subtle emotional differences, making it ideal for mental-health applications.